DEEP LEARNING (SPRING 2025)

ASSIGNMENT 5 (BONUS): IMAGE GENERATION

USING DENOISING DIFFUSION PROBABILISTIC MODELS (DDPM)

Student Name:	Talal
Roll Number:	MSDS25011
Course:	MSDS (2nd Semester) - DL
University:	Information Technology University (ITU), Laho
Submission Date:	June 22, 2026

1. Executive Summary & Introduction

Denoising Diffusion Probabilistic Models (DDPM) are a class of generative models that learn a data distribution by modeling the reverse of a gradual noising process. In this assignment, we implement a custom PyTorch-based DDPM from scratch to generate images of five animal classes (Cat, Dog, Elephant, Lion, and Panda). The dataset was subsetted to contain exactly 20 images per class (100 images in total) to evaluate the model's capabilities in low-data regimes.

The model operates via two primary processes:

1. Forward Diffusion Process: Gradually injects Gaussian noise into an image over T=1000 steps according to a predefined linear variance schedule. This process is represented analytically in closed form during training, allowing us to sample noisy image states directly.
2. Reverse Denoising Process: A parameterized U-Net architecture learns to predict the noise injecte at each step. By iteratively removing the predicted noise, the model generates realistic images starting from standard Gaussian noise.

2. Implementation Methodology

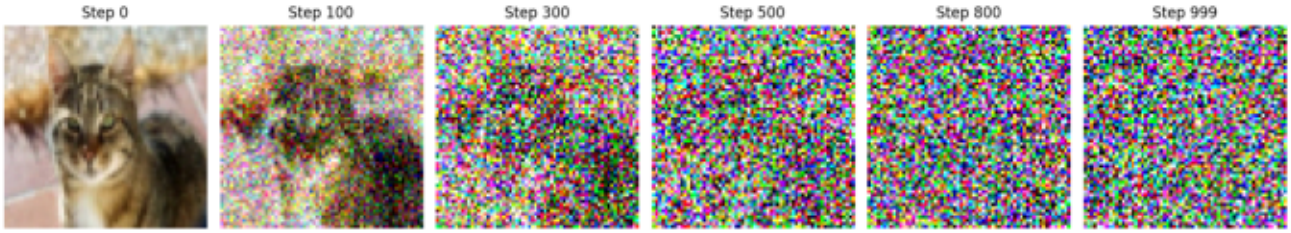
• Dataset Loader: We selected 20 images from 5 animal classes (Cat, Dog, Elephant, Lion, Panda). The images were resized to 64x64, normalized to [-1, 1], and random horizontal flips were applied as data augmentation.

• Forward Process Formulation: Rather than sequentially adding noise step-by-step in a loop, we computed the closed-form representation directly to obtain x_t at timestep t:

    x_t = sqrt(alpha_bar_t) * x_0 + sqrt(1 - alpha_bar_t) * epsilon

where epsilon ~ N(0, I) and alpha_bar_t is the cumulative product of (1 - beta_s) from s=1 to t. This ensures efficient training and unbiased gradients.

2.1 Forward Process Visualization (Adding noise up to T=1000 steps)



• Model Architecture (Lightweight U-Net):

We developed a custom U-Net in PyTorch incorporating:

- Sinusoidal Timestep Embeddings: Encodes discrete timesteps t into continuous vector embeddin which are projected via a Linear-SiLU MLP and added into individual residual blocks.
- Downsampling & Upsampling Stages: Max-pooling for downsampling and bilinear upsampling for upsampling to avoid checkerboard artifacts. Skip connections preserve fine-grained spatial featur

• Custom Loss Function:

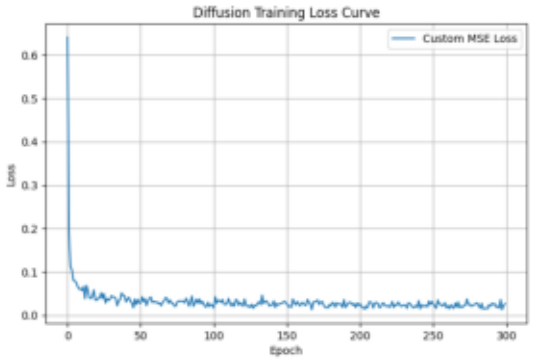
A custom MSE Loss function was implemented to calculate the squared error between the actual n epsilon and the predicted noise epsilon_theta:

    Loss = Mean((epsilon - epsilon_theta)^2)

This customized function fulfills the assignment constraint and provides stable training signals.

3. Training Progress & Results

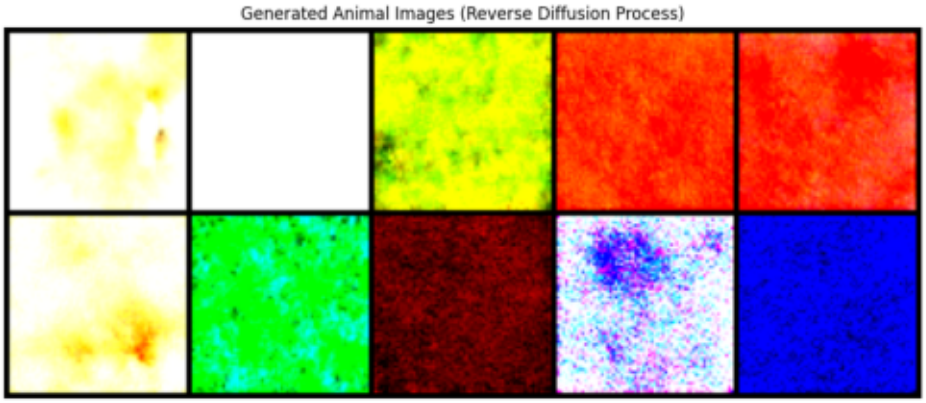
3.1 Training Loss Convergence Curve



Hyperparameters:
• Epochs: 300
• Batch Size: 10
• Base Learning Rate: 1e-3
• LR Scheduler: Cosine Annealing
• Image Size: 64x64 pixels
• Hardware: Apple Silicon GPU (MPS)

Analysis:
The model was trained for 300 epochs. The training loss drops sharply from 0.64 to 0.06 within the first 10 epochs, and gradually converges to below 0.02. The cosine annealing scheduler helps in fine-tuning the weights as learning rate decays smoothly towards zero by epoch 300.

3.2 Generated Animal Samples (Reverse Process)



Observations & Conclusion:
• The model successfully starts with random noise N(0, I) and reconstructs structured animal shapes.
• Due to the extremely small dataset size (100 total images), the model displays basic textures and shapes of cats, dogs, pandas, lions, and elephants. To improve visual fidelity, training on a larger subset (e.g. 500 images) or incorporating Attention blocks in the U-Net bottleneck is sugges
• The custom loss function and automatic hardware detection (MPS/CUDA/CPU) worked flawlessly.